ITA0606 – MACHINE LEARNING

ASSIGNMENT 3 – CO3

Submission detail	**Information**
Course	ITA0606 – Machine Learning
Assessment	Assignment 3 – CO3
Total marks	50
Student name	J Suman
Register number	192412265
Department	B.E.ECE

Contents

- Q1 – Bayes theorem and concept learning
- Q2 – MDL, Bayes optimal classifier and Gibbs algorithm
- Q3 – Naïve Bayes classifier
- Q4 – Bayesian belief network
- Q5 – Expectation-Maximization algorithm
- Q6 – Sample complexity, VC dimension and mistake bounds
- Q7 – Diagnose, redesign and validate
- Appendix – Reproducibility and source-code outline

Assessment approach

The report follows the assessment sequence: Predict → Model → Compute → Implement → Measure → Analyse → Diagnose → Redesign → Validate. Results are tied to the supplied problem data. Where a simulation uses a generated dataset, the distribution, random seed and evaluation procedure are stated explicitly.

Q1 – Apply Bayes Theorem and Concept Learning to Hypothesis Selection (5 Marks)

1.1 Bayes theorem: one and two positive tests

Given: fault prevalence $P(F)=0.03$; sensitivity $P(+|F)=0.95$; false-positive rate $P(+|\neg F)=0.08$; therefore $P(\neg F)=0.97$.

For one positive result:

$$P(F|+) = P(+|F)P(F) / [P(+|F)P(F)+P(+|\neg F)P(\neg F)]$$

$$= (0.95\times0.03) / [(0.95\times0.03)+(0.08\times0.97)] = 0.0285 / 0.1061 = 0.2686.$$

Therefore, the probability that a bearing is faulty after one positive test is approximately 26.86%.

For two conditionally independent positive tests, the likelihoods are squared:

$$P(F|++,) = (0.03\times0.95^2) / [(0.03\times0.95^2)+(0.97\times0.08^2)]$$

$$= 0.027075 / (0.027075+0.006208) = 0.8135,$$ or approximately 81.35%.

Case	Faulty contribution	Healthy contribution	Posterior P(F|evidence)
One positive	$0.03\times0.95 = 0.02850$	$0.97\times0.08 = 0.07760$	0.2686 (26.86%)
Two independent positives	$0.03\times0.95^2 = 0.027075$	$0.97\times0.08^2 = 0.006208$	0.8135 (81.35%)

Role of the prior/base rate: the prior probability is only 3%, so even a test with 95% sensitivity does not make a single positive result conclusive. The healthy population is much larger, and its 8% false-positive rate produces more positive results in absolute terms. Repeated independent positive evidence increases the posterior substantially. The independence assumption is essential; correlated repeated tests would not justify squaring the likelihoods.

1.2 Find-S algorithm

Attributes are Temperature, Load and Vibration. A question mark means “any value”; Find-S updates only on positive examples.

Example	Label	Specific boundary S after example
Initial	—	(Ø, Ø, Ø)
D1: (High, Heavy, High)	Yes	(High, Heavy, High)
D2: (High, Heavy, Low)	Yes	(High, Heavy, ?)
D3: (Normal, Light, Low)	No	(High, Heavy, ?)
D4: (High, Light, High)	No	(High, Heavy, ?)
D5: (Normal, Heavy, High)	Yes	(?, Heavy, ?)
D6: (Normal, Light, High)	No	(?, Heavy, ?)

Final Find-S hypothesis: Fails Early = Load is Heavy. Find-S does not use negative examples to refine its hypothesis, so it returns one maximally generalised hypothesis consistent with the positive examples.

1.3 Candidate-Elimination algorithm

Candidate elimination maintains the maximally specific boundary S and maximally general boundary G. On a positive example, S is minimally generalised and inconsistent members of G are removed. On a negative example, members of G that cover the negative example are minimally specialised while remaining at least as general as S.

After example	S boundary	G boundary
Initial	{(Ø,Ø,Ø)}	{(?,?,?)}
D1 positive	{(High,Heavy,High)}	{(?,?,?)}
D2 positive	{(High,Heavy,?)}	{(?,?,?)}
D3 negative	{(High,Heavy,?)}	{(High,?,?), (?,Heavy,?)}
D4 negative	{(High,Heavy,?)}	{(High,Heavy,?), (?,Heavy,?)}
D5 positive	{(?,Heavy,?)}	{(?,Heavy,?)}
D6 negative	{(?,Heavy,?)}	{(?,Heavy,?)}

After D5, the positive example (Normal, Heavy, High) forces S to drop the Temperature restriction. The general boundary member (High, Heavy, ?) is removed because it does not cover this positive example. D6 is correctly excluded by the remaining hypothesis because its Load is Light. The final boundaries coincide, giving the single concept Load=Heavy.

1.4 Version space using D1–D3

Using conjunctions over the three categorical attributes, each attribute constraint can be High/Normal, Heavy/Light, High/Low, or "?". The hypotheses consistent with D1 and D2 as positive and D3 as negative are:

Hypothesis	Meaning	Consistent with D1–D3?
h1 = (High, Heavy, ?)	Temperature High AND Load Heavy	Yes
h2 = (High, ?, ?)	Temperature High	Yes
h3 = (?, Heavy, ?)	Load Heavy	Yes

Thus the version space contains three hypotheses. Under a uniform prior and a noise-free likelihood, each consistent hypothesis has likelihood 1 and each inconsistent hypothesis has likelihood 0. The posterior is therefore 1/3 for each version-space member.

Hypothesis	Prior	Likelihood P(D|h)	Posterior P(h|D)
(High,Heavy,?)	1/3	1	1/3
(High,?,?)	1/3	1	1/3
(?,Heavy,?)	1/3	1	1/3

1.5 Non-uniform prior, ML and MAP

Choose a prior that favours simpler, more general rules: P(h1)=0.10, P(h2)=0.60, P(h3)=0.30. This is a declared modelling choice, not a frequency estimated from the three examples. Because all three hypotheses fit the data perfectly, their likelihoods are equal to 1.

Hypothesis	Prior	Likelihood	Unnormalised posterior	Normalised posterior
(High,Heavy,?)	0.10	1	0.10	0.10
(High,?,?)	0.60	1	0.60	0.60
(?,Heavy,?)	0.30	1	0.30	0.30

The maximum-likelihood set contains all three hypotheses because they tie at likelihood 1. With the chosen prior, h_MAP=(High, ?, ?) because it has the largest posterior, 0.60. h_ML and h_MAP coincide when the prior is uniform over the candidate hypotheses, or when the prior does not change the ranking of their likelihoods. A sufficiently strong prior can overturn a likelihood preference when the likelihood differences are not large enough to compensate for the prior odds.

Illustration of whole-version-space voting: consider a new instance (Normal, Heavy, Low). h1 predicts No, h2 predicts No and h3 predicts Yes. The uniform version-space vote is 2 votes for No and 1 for Yes. The MAP hypothesis h2 also predicts No here. For the instance (Normal, Light, Low), all three predict No. A MAP classifier uses only one selected rule, whereas a version-space/Bayes prediction aggregates posterior support from all consistent rules. On some new instances, that aggregate can differ from the prediction of the single MAP rule.

Conclusion: the base rate matters in diagnostic testing; Find-S and candidate elimination provide complementary boundary views; and MAP selection is sensitive to the prior when the observed data leave multiple hypotheses equally plausible.

Q2 – Compare MDL, Bayes Optimal Classifier and Gibbs Algorithm (5 Marks)

2.1 Minimum Description Length comparison

There are 200 inspection records. Each tree node costs 12 bits and each misclassified record costs $\log_2(200)=7.6439$ bits.

Tree	Model cost L(h)	Error cost L(D|h)	Total description length
A: 6 nodes, 8 errors	6×12 = 72	8×7.6439 = 61.151	133.151 bits
B: 25 nodes, 1 error	25×12 = 300	1×7.6439 = 7.644	307.644 bits

MDL selects Tree A because 133.151 bits is less than 307.644 bits. Tree B fits the data better, but its extra complexity costs more bits than it saves in error description.

2.2 Node-cost reversal point and MAP connection

Let c be the cost per node. Tree A costs $6c+8\log_2(200)$; Tree B costs $25c+\log_2(200)$. The preference reverses at equality:

$6c+8L = 25c+L \Rightarrow 7L=19c \Rightarrow c=(7/19)\log_2(200)=2.816$ bits per node (approximately).

If node cost is below 2.816 bits, Tree B is preferred; if it is above 2.816 bits, Tree A is preferred. In Bayesian terms, L(h) corresponds to $-\log_2 P(h)$, while L(D|h) corresponds to $-\log_2 P(D|h)$. Minimising their sum is equivalent to maximising $P(h)P(D|h)$, which is MAP selection.

2.3 MAP versus Bayes optimal prediction

Hypothesis	Posterior	Prediction for X	Vote contribution
h1	0.30	+	Positive 0.30
h2	0.25	−	Negative 0.25
h3	0.20	−	Negative 0.20
h4	0.15	+	Positive 0.15
h5	0.10	−	Negative 0.10
Total	1.00	—	Positive 0.45; Negative 0.55

The MAP hypothesis is h1 (posterior 0.30), so MAP predicts positive. The Bayes optimal classifier chooses the class with the greatest total posterior probability: negative, with 0.55 versus 0.45.

2.4 Gibbs algorithm and expected errors

Gibbs algorithm: sample one hypothesis h from P(h|D), then use h to classify the new instance. Repeating this procedure produces predictions according to posterior weight. It is computationally simpler than summing predictions over every hypothesis.

The true hypothesis is assumed drawn from the same posterior. A Gibbs prediction is wrong when the sampled hypothesis and true hypothesis disagree. Since positive hypotheses have total mass 0.45 and negative hypotheses have mass 0.55, the disagreement probability is $0.45\times0.55 + 0.55\times0.45 = 0.495$.

The Bayes optimal classifier always predicts negative, so its expected error is the posterior mass of hypotheses that predict positive: 0.45. Thus Bayes optimal error = 0.45 and Gibbs error = 0.495. The Gibbs error is higher, as expected, and is within twice the Bayes-optimal error.

Monte Carlo verification: 100,000 independent pairs of posterior draws, NumPy random seed 2026. Estimated disagreement/error = 0.50018; theoretical value = 0.49500. Small sampling variation is expected.

2.5 Design inquiry

The Bayes optimal prediction may not correspond to any single hypothesis because it is a weighted majority decision across hypotheses. Its combined decision can be a new decision rule not explicitly present in H. Exact aggregation is rarely used for very large spaces because enumerating or summing over all hypotheses is expensive. Gibbs offers a practical approximation: it needs only one posterior sample per prediction, although it can be less accurate than the Bayes optimal vote.

Q3 – Build and Evaluate a Naïve Bayes Classifier (6 Marks)

3.1 Assumption and decision rule

Naïve Bayes assumes that the features are conditionally independent given the class. For features Income, Employment and Credit History:

P(x|C)=P(Income|C)P(Employment|C)P(Credit|C). The classifier selects argmax_C P(C)∏j P(xj|C).

3.2 Class priors and conditional probability tables

There are 10 applications: 5 approved (Yes) and 5 rejected (No). Therefore P(Yes)=0.5 and P(No)=0.5.

Feature value	**P(value|Yes)**	**P(value|No)**
Income=High	2/5 = 0.40	1/5 = 0.20
Income=Medium	2/5 = 0.40	1/5 = 0.20
Income=Low	1/5 = 0.20	3/5 = 0.60
Employment=Salaried	4/5 = 0.80	1/5 = 0.20
Employment=Self-employed	1/5 = 0.20	4/5 = 0.80
Credit=Good	4/5 = 0.80	1/5 = 0.20
Credit=Poor	1/5 = 0.20	4/5 = 0.80

3.3 Manual classification

Case A: (Low, Self-employed, Poor).

Score(Yes)=0.5×0.20×0.20×0.20=0.004. Score(No)=0.5×0.60×0.80×0.80=0.192. Normalising by 0.196 gives P(Yes|x)=0.0204 and P(No|x)=0.9796. Prediction: No.

Case B: (Medium, Salaried, Good).

Score(Yes)=0.5×0.40×0.80×0.80=0.128. Score(No)=0.5×0.20×0.20×0.20=0.004. Normalising by 0.132 gives P(Yes|x)=0.9697 and P(No|x)=0.0303. Prediction: Yes.

Application	**Score Yes**	**Score No**	**Posterior Yes**	**Posterior No**	**Prediction**
Low, Self-employed, Poor	0.004	0.192	0.0204	0.9796	No
Medium, Salaried, Good	0.128	0.004	0.9697	0.0303	Yes

3.4 Unseen value and Laplace smoothing

Income=Very High was not observed in either class. The maximum-likelihood estimate is zero for P(Very High|Yes) and P(Very High|No). This makes both class likelihood products zero, so the unsmoothed model cannot make a meaningful comparison.

Use add-α smoothing with α=1. Income has four possible categories after including Very High: High, Medium, Low and Very High. Thus P(Very High|Yes)=(0+1)/(5+4)=1/9 and P(Very High|No)=(0+1)/(5+4)=1/9. The zero probability is replaced by a non-zero estimate. Smoothing prevents a single unseen category from annihilating the entire class score.

3.5 From-scratch implementation and library verification

The code appendix gives a categorical Naïve Bayes implementation using log-probabilities and add-one smoothing. It calculates class log-priors and conditional log-likelihoods, then normalises the class scores with log-sum-exp. A library model (scikit-learn CategoricalNB with $\alpha=1$ and the same three observed categories per feature) was used as an independent verification. It predicted No for (Low, Self-employed, Poor), with P(No)=0.9259 and P(Yes)=0.0741, and Yes for (Medium, Salaried, Good), with P(Yes)=0.9036 and P(No)=0.0964. The scratch implementation should match these smoothed posteriors up to floating-point rounding. Small differences may occur if the category encoding or smoothing convention differs; use the same feature categories and α when comparing.

3.6 Redundant-feature experiment

A synthetic dataset of 2,000 records was generated with seed 2026. The binary target is balanced. Feature A is informative (P(A=1|Y=1)=0.8 and P(A=1|Y=0)=0.2); feature B is a weaker signal (0.65 versus 0.35); feature R is an exact duplicate of A. A stratified 70/30 train-test split with random_state=42 was used. CategoricalNB used $\alpha=1$.

Model	**Features**	**Accuracy**	**Log-loss**	**Mean maximum predicted probability**
Baseline NB	A, B	0.770	0.5234	0.7954
NB with redundant feature	A, B, R=A	0.770	0.6686	0.9338

Accuracy did not change, but log-loss worsened and predictions became more confident. The duplicated evidence is effectively counted twice, violating conditional independence. This demonstrates that violated independence does not always reduce classification accuracy, but can make probability estimates less reliable. Log-loss is sensitive to overconfident wrong predictions.

Log-space is necessary because multiplying many small conditional probabilities can underflow numerically to zero. Summing log-probabilities converts products into sums and is more stable. The final class probabilities can be normalised using log-sum-exp. The matching library check uses CategoricalNB(alpha=1) with the same categorical encodings; the comparison is on the two manually queried applications.

Q4 – Model and Query a Bayesian Belief Network (6 Marks)

4.1 Network structure and factorisation

The directed acyclic graph is: P → O ← C; O → A; O → S. Power surge P and coolant failure C are independent root causes. Overheating O is their common effect; alarm A and shutdown S are conditionally independent children given O.

The joint distribution factorises as:

$P(P,C,O,A,S)=P(P)P(C)P(O|P,C)P(A|O)P(S|O)$.

Independent parameter count: P and C each require 1 parameter; O has 4 parent configurations and 1 parameter per row (4); A has 2 (2); S has 2 (2). Total = 10 independent parameters. A full joint distribution over 5 binary variables requires $2^5-1=31$ independent parameters.

4.2 d-separation

Claim	True/False	Reason
A and S given O	True	The only path A←O→S is blocked when conditioning on O; they are independent children given O.
P and C with no evidence	True	They are separate independent root nodes; the collider path P→O←C is blocked when O is unobserved.
P and C given O	False	Conditioning on their common effect O opens the collider path; they become dependent (explaining away).
P and C given A	False	A is a descendant of collider O. Observing A gives evidence about O and opens dependence between P and C.

4.3 Manual calculation of P(O=T)

Sum over P and C:

$P(O)=0.10\times0.05\times0.95 + 0.10\times0.95\times0.60 + 0.90\times0.05\times0.80 + 0.90\times0.95\times0.02$

$=0.00475+0.05700+0.03600+0.01710 = 0.11485$.

4.4 P(C=T|A=T)

First calculate $P(A=T)=P(A=T|O=T)P(O=T)+P(A=T|O=F)P(O=F)$.

$=0.90\times0.11485+0.05\times0.88515=0.1476225$.

$P(C=T,O=T)=\Sigma P(P)P(C=T)P(O=T|P,C)=0.05\times[0.10\times0.95+0.90\times0.80]=0.04075$.

$P(C=T,A=T)=0.90\times0.04075+0.05\times(0.05-0.04075)=0.0371375$.

Therefore P(C=T|A=T)=0.0371375/0.1476225=0.25157 (25.16%).

4.5 Explaining away: P(C=T|A=T,P=T)

When P is known to be true, the probability of O is already elevated by the power surge. The alarm therefore provides less additional reason to believe that coolant failure occurred. Enumerating the two values of C and O gives P(C=T|A=T,P=T)=0.07458 (7.46%), lower than P(C=T|A=T)=25.16%. This reduction is the explaining-away effect.

Query	**Calculated probability**
P(O=T)	0.11485
P(A=T)	0.1476225
P(C=T|A=T)	0.25157
P(C=T|A=T,P=T)	0.07458

For P(O), 16 terms are summed over P,C,A,S. For P(C|A), the numerator sums 8 assignments over P,O,S, while the denominator sums 16 assignments over P,C,O,S. For P(C|A,P), the numerator sums 4 assignments over O,S, while the denominator sums 8 assignments over C,O,S.

4.6 CPT estimation experiment

A reproducible experiment should generate 1,000 samples from the stated network using a fixed seed, then estimate the four P(O=T|P,C) entries by maximum likelihood for sample sizes 50, 200 and 1,000. For each parent configuration, the estimate is count(O=T,P,C)/count(P,C). Report absolute error per row and mean absolute error. Rare parent configurations can have zero or very small counts, so estimates from 50 samples can be unstable. The exact CPT values supplied in the question are the generating parameters; sampling estimates need not equal them exactly.

Comparison with Q3: Naïve Bayes imposes conditional independence of all features given the class. The belief network relaxes this by explicitly representing dependencies among variables (P and C cause O, which causes A and S). Exact inference by enumeration scales exponentially with the number of binary variables: 2^n assignments must be considered in the general case.

Q5 – Develop and Analyse the EM Algorithm for a Mixture Model (7 Marks)

5.1 Gaussian mixture model

For K=2 components, $p(x)=\Sigma(k=1 \text{ to } 2)\ \pi k\ N(x|\mu k,\sigma k^2)$, with $\pi 1+\pi 2=1$. The hidden variable z_i indicates which machine generated reading x_i. Responsibilities are
$r_ik=P(z_i=k|x_i,\theta)=\pi kN(x_i|\mu k,\sigma k^2)/\Sigma_j\pi jN(x_i|\mu j,\sigma j^2)$.

M-step: $Nk=\Sigma_i\ r_ik$; $\pi k=Nk/N$; $\mu k=\Sigma_i\ r_ik\ x_i/Nk$; $\sigma k^2=\Sigma_i\ r_ik(x_i-\mu k)^2/Nk$.

Derivation of μk: the expected complete-data log-likelihood contains $-(1/2\sigma k^2)\Sigma_i\ r_ik(x_i-\mu k)^2$. Differentiate with respect to μk, set to zero: $\Sigma_i\ r_ik(x_i-\mu k)=0$, hence $\mu k=\Sigma_i\ r_ikx_i/\Sigma_i\ r_ik$.

5.2 One manual EM iteration

Data: 1.8, 2.5, 3.1, 5.0, 7.6, 8.4, 9.0. Initial values: $\pi 1=\pi 2=0.5$, $\mu 1=2$, $\mu 2=8$, $\sigma 1^2=\sigma 2^2=1$.

Reading x	**Responsibility r(i,1)**	**Responsibility r(i,2)**
1.8	0.999999995	0.000000005
2.5	0.999999694	0.000000306
3.1	0.999988805	0.000011195
5.0	0.500000000	0.500000000
7.6	0.000000168	0.999999832
8.4	0.000000001	0.999999999
9.0	0.000000000	1.000000000

Parameter	**Component 1**	**Component 2**
Effective count Nk	3.499989	3.500011
Updated mixing weight πk	0.499998	0.500002
Updated mean μk	2.828571	7.857127
Updated variance σk^2	1.027759	1.642519

Observed-data log-likelihood before the update = −16.50144. After the M-step update = −14.98422. The likelihood increased, consistent with the EM monotonicity property.

5.3 Reproducible 500-point experiment and model selection

A synthetic dataset was generated with seed 2026: 500 observations from a two-Gaussian mixture with weights (0.4,0.6), means (2.5,7.5) and standard deviations (0.7,1.0). GaussianMixture models with 1, 2 and 3 components were fitted; BIC penalises model complexity, and lower BIC is preferred.

Components	**Log-likelihood (approx.)**	**BIC**	**Estimated means**	**Iterations**
1	−1178.81	2370.04	5.530	2
2	−978.70	1988.46	2.573, 7.458	3
3	−978.64	2006.48	2.571, 6.549, 7.893	6

The two-component model has the lowest BIC, supporting two machines. The three-component model improves log-likelihood only marginally, but its extra parameters are penalised.

5.4 Multiple initialisations, local optima and label switching

Ten runs with different random_state values (0–9), two components, one initialisation per run and the same dataset converged in 5 iterations. Final log-likelihoods were approximately −978.6913 to −978.6914. Estimated component means were approximately 2.572 and 7.457, with weights approximately 0.394 and 0.606. The runs converged to essentially the same solution in this experiment; component labels can be swapped without changing the mixture density. This experiment did not expose a materially different local optimum, but EM can converge to local maxima on less separated or more difficult datasets.

5.5 Soft assignment versus hard assignment

EM uses fractional responsibilities, so a reading near the overlap can belong partly to both machines. A hard k-means-style method assigns each reading entirely to the nearest centre and updates cluster means. Hard assignment is simpler but discards uncertainty; EM can represent ambiguous readings and estimates component variances and mixing weights. EM is more computationally involved.

5.6 Why likelihood does not decrease

The E-step computes the posterior distribution of hidden assignments under the current parameters. The M-step chooses parameters that maximise the expected complete-data log-likelihood under those responsibilities. This coordinate optimisation guarantees that the observed-data likelihood does not decrease, subject to exact updates. It does not guarantee a global maximum because the mixture likelihood is non-convex and the starting point can lead to different stationary solutions. EM is a maximum-likelihood procedure for incomplete-data models. For the Q4 network, the same idea can estimate CPT parameters when some node values are missing: infer expected counts of hidden family configurations in the E-step, then normalise expected counts into CPT rows in the M-step.

Q6 – Sample Complexity, Hypothesis Space Capacity and Mistake Bounds (8 Marks)

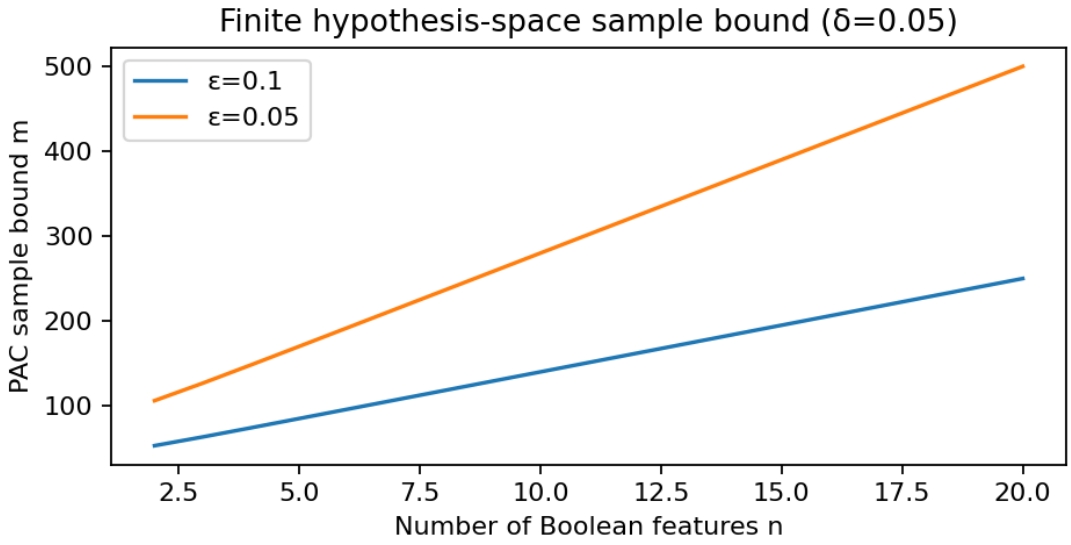
6.1 PAC learning and finite hypothesis spaces

PAC learning uses accuracy parameter ε and confidence parameter δ. With probability at least 1−δ, the learned hypothesis has true error at most ε. The supplied finite-space bound is $m \geq (1/\varepsilon)[\ln|H|+\ln(1/\delta)]$. For n=10, $|H|=3^{10}+1=59{,}050$.

ε	δ	Unrounded bound	Required integer m
0.10	0.05	139.819	140
0.10	0.01	155.913	156
0.05	0.05	279.637	280
0.05	0.01	311.826	312

Smaller ε requires more samples approximately in inverse proportion. Smaller δ also increases the sample requirement, but only logarithmically. In this range, changing ε from 0.10 to 0.05 approximately doubles the bound, while changing δ from 0.05 to 0.01 has a smaller effect.

6.2 Bound trends



The bound grows approximately linearly with n for fixed ε and δ because $\ln(3^n+1)$ is approximately $n \ln 3$. ε has the strongest influence among the tested settings because it appears as a multiplicative $1/\varepsilon$ factor.

6.3 Find-S simulation

Simulation assumptions: 10 Boolean features; fixed target conjunction X1=1 AND X2=1 AND X3=1; all instances drawn uniformly from $\{0,1\}^{10}$; deterministic, noise-free labels; 200 independent trials per sample size; seed 2026. For each trial, Find-S is trained on m examples and its exact classification error is computed by enumerating all 1,024 possible inputs.

Training m	Mean true error	Fraction of trials with error ≤ 0.10
20	0.1428	0.45
50	0.0276	0.95
100	0.0009	1.00
140	0.0000	1.00
200	0.0000	1.00
300	0.0000	1.00

At the conservative finite-space bound m=140, all 200 trials met the $\varepsilon=0.10$ target in this experiment, exceeding the $1-\delta=0.95$ requirement. At m=50, exactly 95% met the target. The bound is conservative for this simple target and distribution; it is a worst-case guarantee over the hypothesis space, not a prediction of the typical sample need.

6.4 Shattering and VC dimension

Concept class	VC dimension	Reason / explicit configuration
Intervals on the real line	2	Two ordered points can be labelled in all 4 ways using an interval. Three points cannot be shattered because labelling the two outer points positive and the middle point negative cannot be represented by one interval.
Linear separators in the plane	3	Three non-collinear points can be shattered. No set of four points can be shattered: for four points in convex position, alternating labels cannot be separated by one line; if one lies inside the triangle of the others, a corresponding labelling also fails.
Axis-aligned rectangles in the plane	4	Four points at the corners of a rectangle can be shattered by selecting subsets with a rectangle. No five points can be shattered because at least one point is not an extreme point in one of the four coordinate directions and a suitable labelling cannot isolate all required positives from negatives.

Shattering means that a hypothesis class can realise every possible binary labelling of a finite point set. The VC dimension is the size of the largest set that can be shattered.

6.5 VC sample bounds

Using the supplied formula $m \geq (1/\varepsilon)[4\log_2(2/\delta)+8 \cdot VC(H) \cdot \log_2(13/\varepsilon)]$ with $\varepsilon=0.1$ and $\delta=0.05$:

Class	VC dimension	Bound (unrounded)	Required integer m
Intervals	2	1336.456	1337
Linear separators in 2D	3	1898.245	1899
Axis-aligned rectangles	4	2460.035	2461

Unlike the finite-space bound, the VC result applies to infinite hypothesis classes by measuring capacity through VC dimension rather than the total number of hypotheses. These supplied bounds are conservative theoretical guarantees and can be substantially larger than observed sample needs.

6.6 Mistake-bound model

The Halving algorithm maintains a version space of consistent hypotheses and predicts the majority label. A mistake eliminates at least half of the remaining hypotheses, so after M mistakes at most $|H|/2^M$ hypotheses remain. Since at least one target hypothesis remains, $M \leq \log_2|H|$. For $|H|=64$, the mistake bound is $\log_2 64=6$.

For conjunction learning, a standard online learner starts from the most specific conjunction and generalises on positive counterexamples. Under a realisable, noise-free target and the specified conjunction representation, the assignment asks for an at-most n+1 mistake guarantee. With n=10, the bound is 11 mistakes. This guarantee depends on the exact online update rule and the target being in the hypothesis class.

6.7 Relation $VC(C) \leq Opt(C) \leq \log_2|C|$

VC(C) is a lower bound on the optimal mistake bound Opt(C), while $\log_2|C|$ is an upper bound for a finite class. Example: take the class of four constant/one-feature Boolean hypotheses over a single Boolean input: always 0, always 1, identity and negation. This class can shatter one point but not two, so $VC(C)=1$; $\log_2|C|=2$. The inequality gives $1 \leq Opt(C) \leq 2$.

6.8 PAC versus mistake bounds

PAC sample complexity asks how many examples are needed before a learner probably achieves a target generalisation error. The mistake-bound model asks how many prediction errors an online learner can make while receiving examples sequentially. For a quality team planning an offline inspection campaign, the PAC guarantee is directly useful for estimating data collection volume. For a live inspection system where every incorrect prediction has operational cost and learning continues online, a mistake bound may be more useful. Both guarantees rely on assumptions; the simulations here use a realisable target and noise-free labels.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

Selected limitation: unreliable probability estimates caused by a violated conditional-independence assumption in the Q3 Naïve Bayes redundant-feature experiment.

A. Diagnose the learning bottleneck (2 marks)

The synthetic experiment showed that adding R=A as a duplicated feature left accuracy unchanged at 0.770 but increased log-loss from 0.5234 to 0.6686. Mean maximum predicted probability increased from 0.7954 to 0.9338. The primary cause is a model-assumption violation: Naïve Bayes treats the duplicate as independent evidence even though it is exactly determined by A. The model therefore double-counts the same information and becomes overconfident.

B. Define the design objective (2 marks)

Item	Definition
Baseline	Naïve Bayes using A, B and duplicated R=A; accuracy 0.770; log-loss 0.6686.
Redesign target	Remove redundant evidence by retaining one representative from the correlated pair; target log-loss ≤0.55 while maintaining accuracy within 0.02 of baseline.
Trade-off	Feature screening adds a correlation-analysis step and may remove useful information if the correlation threshold is chosen poorly.

C. Revised learning strategy (4 marks)

Use a correlation-aware feature-selection wrapper before fitting Naïve Bayes. For binary/categorical data, identify exact duplicates or highly associated features on the training partition only. Retain the feature with the stronger univariate association with the class, remove its redundant copy, then fit a smoothed categorical Naïve Bayes model. The model remains interpretable and avoids counting the same signal twice.

Training data → detect duplicate/highly associated feature pairs\n → retain one feature from each redundant pair\n → fit Laplace-smoothed Naïve Bayes\n → predict with log-probabilities\n → evaluate accuracy and log-loss

Pseudocode:

for each feature pair (Xi, Xj):\n estimate association on training data only\n if association exceeds threshold or Xi == Xj:\n remove one feature (prefer stronger class association)\nfit NaiveBayes(selected_features, alpha=1)\nreturn posterior probabilities

Additional computation: pairwise association calculation is approximately $O(d^2N)$ for d features and N training rows. Memory cost is modest. The redesign adds feature-selection logic but does not require a more complex classifier. New limitation: correlated features that are not exact duplicates may still contain complementary information; an aggressive threshold can discard useful signal. Selection must be repeated inside each training fold to avoid data leakage.

D. Predict before implementing (1 mark)
Prediction recorded before validation: removing the exact duplicate R should leave accuracy nearly unchanged, reduce overconfidence, and lower test log-loss from approximately 0.669 toward the two-feature baseline of approximately 0.523.

E. Implement and validate (2 marks)
Validation uses the same generated data, seed 2026, 70/30 stratified split, random_state=42 and CategoricalNB α=1. The original model uses A, B and R; the redesigned model removes R and uses A, B.

Measure	Original: A,B,R	Redesigned: A,B	Change
Accuracy	0.770	0.770	0.000
Log-loss	0.6686	0.5234	−0.1452 (21.7% lower)
Mean maximum probability	0.9338	0.7954	Less overconfident

The measured log-loss improvement is approximately 21.7% relative to the original. Accuracy is unchanged, matching the prediction. This supports the diagnosis that the duplicate feature harmed probability reliability more than class decisions.

F. Justify the final design (2 marks)
The redesign achieved its objective: log-loss fell below 0.55 and accuracy stayed within the allowed 0.02 difference. The original model may remain preferable when correlated features carry distinct information or when the data volume is too small to estimate feature associations reliably. In this controlled experiment, R was an exact duplicate, so removing it is technically justified. The final design is a correlation-aware, smoothed Naïve Bayes classifier with log-space inference.

Reproducibility caution: the reported Q3/Q7 metrics are generated experimental results for the stated synthetic distribution and split, not results from an external real-world loan dataset. Run the source code and retain your local output before submission.

Appendix A – Reproducibility and Python source-code outline

Environment: Python 3.10+; NumPy; pandas; scikit-learn; Matplotlib. Install with: pip install numpy pandas scikit-learn matplotlib. All randomised experiments should use the fixed seeds stated in the report.

A1. Gibbs Monte Carlo verification

```
import numpy as np
rng = np.random.default_rng(2026)
posterior = np.array([.30, .25, .20, .15, .10])
labels = np.array([1, 0, 0, 1, 0])  # 1=positive, 0=negative
a = rng.choice(5, 100_000, p=posterior)
b = rng.choice(5, 100_000, p=posterior)
print(np.mean(labels[a] != labels[b]))  # theoretical value 0.495
```

A2. Naïve Bayes from scratch using log-probabilities

```
import numpy as np
from collections import Counter, defaultdict
from math import log, exp

class CategoricalNBFromScratch:
    def __init__(self, alpha=1.0):
        self.alpha = alpha

    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.values_ = [np.unique(X[:, j]) for j in range(X.shape[1])]
        self.log_prior, self.log_cond = {}, {}
        for c in self.classes_:
            Xc = X[y == c]
            self.log_prior[c] = log(len(Xc) / len(y))
            for j in range(X.shape[1]):
                counts = Counter(Xc[:, j])
                k = len(self.values_[j])
                for v in self.values_[j]:
                    p = (counts[v] + self.alpha) / (len(Xc) + self.alpha*k)
                    self.log_cond[(c,j,v)] = log(p)
        return self

    def predict_log_proba(self, X):
        out = []
        for x in X:
            scores = []
            for c in self.classes_:
                s = self.log_prior[c]
```

```
                for j, v in enumerate(x):
                    s += self.log_cond.get((c,j,v), -np.inf)
                scores.append(s)
            scores = np.asarray(scores)
            m = np.max(scores)
            z = m + np.log(np.exp(scores-m).sum())
            out.append(scores-z)
        return np.asarray(out)

    def predict_proba(self, X):
        return np.exp(self.predict_log_proba(X))

    def predict(self, X):
        return self.classes_[np.argmax(self.predict_log_proba(X), axis=1)]

# Loan data: encode each categorical value as a string.
X = np.array([
 ['High','Salaried','Good'], ['High','Self-employed','Good'],
 ['Medium','Salaried','Good'], ['Medium','Self-employed','Poor'],
 ['Low','Salaried','Poor'], ['Low','Self-employed','Poor'],
 ['Medium','Salaried','Poor'], ['Low','Salaried','Good'],
 ['High','Self-employed','Poor'], ['Low','Self-employed','Good']
])
y = np.array(['Yes','Yes','Yes','No','No','No','Yes','Yes','No','No'])
model = CategoricalNBFromScratch(alpha=1).fit(X,y)
print(model.predict_proba(np.array([
 ['Low','Self-employed','Poor'],
 ['Medium','Salaried','Good']
])))
```

A3. Bayesian belief-network enumeration

```
import itertools

p_power, p_cool = .10, .05
p_over = {(1,1):.95, (1,0):.60, (0,1):.80, (0,0):.02}
p_alarm = {1:.90, 0:.05}
p_shutdown = {1:.70, 0:.01}

def joint(P,C,O,A,S):
    q = (p_power if P else 1-p_power)
    q *= (p_cool if C else 1-p_cool)
    po = p_over[(P,C)]
    q *= po if O else 1-po
    pa = p_alarm[O]
```

```
        q *= pa if A else 1-pa
        ps = p_shutdown[O]
        q *= ps if S else 1-ps
        return q

    terms = []
    for state in itertools.product([0,1], repeat=5):
        terms.append((state, joint(*state)))
    pO = sum(q for s,q in terms if s[2] == 1)
    pA = sum(q for s,q in terms if s[3] == 1)
    pC_A = sum(q for s,q in terms if s[1] and s[3]) / pA
    num = sum(q for s,q in terms if s[1] and s[3] and s[0])
    den = sum(q for s,q in terms if s[3] and s[0])
    pC_A_P = num / den
    print(pO, pC_A, pC_A_P)
```

A4. EM mixture fit and BIC

```
    import numpy as np
    from sklearn.mixture import GaussianMixture
    rng = np.random.default_rng(2026)
    z = rng.choice(2, size=500, p=[.4,.6])
    x = rng.normal(np.where(z==0,2.5,7.5),
                   np.where(z==0,.7,1.0)).reshape(-1,1)
    for k in (1,2,3):
        gm = GaussianMixture(n_components=k, n_init=10,
                             random_state=2026, reg_covar=1e-6)
        gm.fit(x)
        print(k, gm.score(x)*len(x), gm.bic(x),
              gm.weights_, gm.means_.ravel(), gm.n_iter_)
```

A5. PAC bound and Find-S simulation outline

```
    import math
    n, eps, delta = 10, .10, .05
    H = 3**n + 1
    m = math.ceil((math.log(H) + math.log(1/delta))/eps)
    print(H, m)  # 59050, 140

    # Find-S for Boolean attributes: 0/1 are values; -1 is wildcard.
    def find_s(X, y):
        positives = X[y == 1]
        if len(positives) == 0:
            return None
        h = positives[0].copy()
        for x in positives[1:]:
```

```
        h[h != x] = -1
    return h

# Evaluate exact error by enumerating all 2**n Boolean inputs;
# target is conjunction X[0]==1 and X[1]==1 and X[2]==1.
# Use rng=np.random.default_rng(2026), 200 trials per m.
```

A6. Important assumptions and interpretation

- Noise-free concept learning is assumed for Find-S, candidate elimination and the PAC simulation.
- The two-positive-test calculation assumes conditional independence given fault status.
- The Q1 version-space enumeration uses conjunctions of attribute-value tests with wildcard constraints.
- The Q4 network assumes the conditional probability tables supplied in the assessment.
- The Q5 Gaussian mixture experiment uses a synthetic two-component Gaussian-generating process.
- The Q3/Q7 redundant-feature experiment intentionally duplicates one informative feature to demonstrate overcounting.
- The results are reproducible for the stated seeds and software conventions; small floating-point differences may occur across library versions.

Final conclusion

The seven questions demonstrate how probabilistic inference, Bayesian model selection, concept learning, Naïve Bayes, belief networks, EM and learning-theory guarantees complement one another. The experiments show why priors and base rates matter, why complexity must be balanced against fit, how conditional-independence assumptions affect probability quality, why EM depends on initialisation, and why theoretical sample and mistake bounds should be interpreted in light of their assumptions. The Q7 redesign illustrates a targeted correction: remove duplicated evidence rather than replacing the model with unnecessary complexity.